

Research Notes on Bagging ensemble

Bagging ensemble

>> Section 1

Specify the maximum depth of trees: Instead of allowing the random forest to continue until all nodes are pure, it is better to cut it off at a certain point in order to further decrease chances of overfitting. Prune the dataset: Using an extremely large dataset may create results that are less indicative of the data provided than a smaller set that more accurately represents what is being focused on. Continue pruning the data at each node split rather than just in the original bagging process. Decide on accuracy or speed: Depending on the desired results, increasing or decreasing the number of trees within the forest can help. Increasing the number of trees generally provides more accurate results while decreasing the number of trees will provide quicker results.

>> Section 2

Creation of Decision Trees The next step of the algorithm involves the generation of decision trees from the bootstrapped dataset. To achieve this, the process examines each gene/feature and determines for how many samples the feature's presence or absence yields a positive or negative result. This information is then used to compute a confusion matrix, which lists the true positives, false positives, true negatives, and false negatives of the feature when used as a classifier. These features are then ranked according to various classification metrics based on their confusion matrices. Some common metrics include estimate of positive correctness (calculated by subtracting false positives from true positives), measure of "goodness", and information gain. These features are then used to partition the samples into two sets: those that possess the top feature, and those that do not. The diagram below shows a decision tree of depth two being used to classify data. For example, a data point that exhibits Feature 1, but not Feature 2, will be given a "No". Another point that does not exhibit Feature 1, but does exhibit Feature 3, will be given a "Yes".

>> Section 3

By taking the average of 100 smoothers, each corresponding to a subset of the original dataset, we arrive at one bagged predictor (red line). The red line's flow is stable and does not overly conform to any data point(s).

>> Section 4

For a weak learner with high bias, bagging will also carry high bias into its aggregate. Loss of interpretability of a model. Can be computationally expensive depending on the dataset.

>> Section 5

Because of their properties, random forests are considered one of the most accurate data mining algorithms, are less likely to overfit their data, and run quickly and efficiently even for large datasets. They are primarily useful for classification as opposed to regression, which attempts to draw observed connections between statistical variables in a dataset. This makes random forests particularly useful in such fields as banking, healthcare, the stock market, and e-commerce where it is important to be able to predict future results based on past data. One of their applications would be as a useful tool for predicting cancer based on genetic factors, as seen in the above example. There are several important factors to consider when designing a random forest. If the trees in the random forests are too deep, overfitting can still occur due to over-specificity. If the forest is too large, the algorithm may become less efficient due to an increased runtime. Random forests also do not generally perform well when given sparse data with little variability. However, they still have numerous advantages over similar data classification algorithms such as neural networks, as they are much easier to interpret and generally require less data for training. As an integral component of random forests, bootstrap aggregating is very important to classification algorithms, and provides a critical element of variability that allows for increased accuracy when analyzing new data, as discussed below.

>> Section 6

Bagging leads to "improvements for unstable procedures", which include, for example, artificial neural networks, classification and regression trees, and subset selection in linear regression. Bagging was shown to improve preimage learning. On the other hand, it can mildly degrade the performance of stable methods such as k-nearest neighbors.

>> Section 7

Bootstrap aggregating, also called bagging (from bootstrap aggregating) or bootstrapping, is a machine learning (ML) ensemble meta-algorithm designed to improve the stability and accuracy of ML classification and regression algorithms. It also reduces variance and overfitting. Although it is usually applied to decision tree methods, it can be used with any type of method. Bagging is a special case of the ensemble averaging approach.

>> Section 8

Improving Random Forests and Bagging While the techniques described above utilize random forests and bagging (otherwise known as bootstrapping), there are certain techniques that can be used in order to improve their execution and voting time, their prediction accuracy, and their overall performance. The following are key steps

Research Notes on Bagging ensemble

Bagging ensemble

in creating an efficient random forest:

>> Section 9

History The concept of bootstrap aggregating is derived from the concept of bootstrapping which was developed by Bradley Efron. Bootstrap aggregating was proposed by Leo Breiman who also coined the abbreviated term "bagging" (bootstrap aggregating). Breiman developed the concept of bagging in 1994 to improve classification by combining classifications of randomly generated training sets. He argued, "If perturbing the learning set can cause significant changes in the predictor constructed, then bagging can improve accuracy".

>> Section 10

Application Creating the bootstrap and out-of-bag datasets is crucial since it is used to test the accuracy of ensemble learning algorithms like random forest. For example, a model that produces 50 trees using the bootstrap/out-of-bag datasets will have a better accuracy than if it produced 10 trees. Since the algorithm generates multiple trees and therefore multiple datasets the chance that an object is left out of the bootstrap dataset is low. The next few sections talk about how the random forest algorithm works in more detail.

>> Section 11

This process is repeated recursively for successive levels of the tree until the desired depth is reached. At the very bottom of the tree, samples that test positive for the final feature are generally classified as positive, while those that lack the feature are classified as negative. These trees are then used as predictors to classify new data.